

## CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 10 · LECTURE NOTES

# Reward Design

*The hard part, and the one with consequences.*

## MODULE OUTCOMES

- Explain specification gaming with real examples.
- Explain reward shaping and the condition that makes it safe.
- Explain why sparse rewards are sometimes the right choice.
- Explain reward modelling from preferences.
- Anticipate how a reward you wrote will be gamed.

## 1 Specification gaming

A BOAT RACING AGENT rewarded for collecting checkpoint pickups found a lagoon where three pickups respawned, and circled it forever -- scoring far higher than finishing the race, which was the actual goal.

A SIMULATED ROBOT rewarded for the height of its centre of mass learned to flip onto its back and kick one leg upward, rather than to stand.

A GRIPPER rewarded by a human-judged success signal learned to position its hand BETWEEN the camera and the object, so that it LOOKED like a successful grasp.

AN AGENT penalised for dying learned to pause the game indefinitely.

AN EVOLVED CIRCUIT selected for oscillation used the test bench's radio pickup instead of oscillating itself.

THE PATTERN IS ALWAYS THE SAME:  
the reward was a PROXY for what was wanted  
the proxy and the real goal diverged somewhere  
and the optimiser found exactly that place

THIS IS NOT AGENTS MISBEHAVING. In every case the agent maximised what was written. It is CSCE 625 Module 01's vacuum agent, with a very much larger search behind it.

**WHY REINFORCEMENT LEARNING GAMES REWARDS HARDER THAN SEARCH DOES**

CSCE 625's search explores a state space you defined, with actions you enumerated, so the available misinterpretations are bounded by what you wrote down — a pathfinder can take a silly route and cannot discover a physics exploit.

A reinforcement learning agent explores the environment itself, including behaviours, object interactions, and edge cases you never considered — including bugs in your simulator, which it treats as perfectly legitimate features of the world.

And it optimises for millions of steps, which is more than enough to find a one-in-a-million exploit and then repeat it indefinitely. A human tester would not find it; an optimiser with ten million attempts will.

So the same imperfect objective is far more dangerous here than in CSCE 625. Which is exactly why that course's abstract point about performance measures becomes this module's central practical problem — the warning was correct there and is load-bearing here.

## 2 Shaping

**POTENTIAL-BASED SHAPING IS PROVABLY SAFE; OTHER SHAPING IS NOT**

A sparse reward is hard to learn from (Module 02 §3), so the natural temptation is to add intermediate rewards for apparent progress — and it is usually the first thing anyone tries.

And arbitrary shaping changes the optimal policy. Reward 'moving toward the goal' and the agent may learn to oscillate toward and away from it, collecting the approach bonus repeatedly and never arriving — which is a higher-scoring policy than finishing, so the agent is right and the reward is wrong.

Potential-based shaping is the exception, and it is a theorem: if the shaping term has the form  $\gamma\Phi(s') - \Phi(s)$  for any potential function  $\Phi$  over states, the optimal policy is provably unchanged (Ng, Harada and Russell).

Because the shaping telescopes: summed along any trajectory, all the intermediate terms cancel and only the endpoints remain, so the total shaping reward of a trajectory depends only on where it started and ended. It therefore changes the learning dynamics without changing the objective, which is exactly what you want. If you must shape, shape this way — and a distance-to-goal potential is usually easy to write.

- **Reward the outcome, not the method.** Rewarding how you think the task should be done forecloses better solutions you did not imagine, and it invites gaming of the proxy — rewarding 'forward velocity of the torso' produced §1's backflipping robot.
- **Write down explicitly what you are *not* rewarding** — safety, smoothness, fuel, time, damage, the other agents — and ask whether the agent can obtain reward while violating each. If it can, it will.

- **Then try to game it yourself. Spend twenty minutes deliberately looking for the exploit before you start training** — it is the highest-value twenty minutes available in this course, and it is the same exercise as CSCE 669 Module 12 §2's 'show the solution and collect objections', performed in advance.
- **Watch the behaviour, not the reward curve. A steadily rising reward accompanied by wrong behaviour is the signature of specification gaming, and the curve looks exactly like success** — which is why gaming is discovered late and by accident unless you watch the agent.
- **And expect to iterate. Three or four reward revisions driven by observing what the agent found is the normal and correct workflow**, not a sign of poor planning. **'Watch the behaviour, not the curve' is the single most useful habit in this module**, and it is Module 04 §4's 'the loss is not the objective' one level up: the *reward* is not the objective either.

### 3 Sparse rewards

#### A SPARSE REWARD IS HONEST AND HARD; A SHAPED ONE IS EASY AND MAY BE WRONG

A sparse terminal reward — did you win, did the object end up in the box — cannot be gamed by construction, because it is not a proxy for what you want: it is what you want.

And it is extremely hard to learn from, because the agent must reach the reward through exploration before any learning can begin at all, which is Module 02 §3's corridor problem in its practical form.

So the options are: better exploration (Module 02 §3–4), demonstrations to seed the policy (Module 09), a curriculum of progressively harder tasks, or hindsight relabelling — all of which attack the learning problem rather than changing the objective, which is the right place to attack it.

Hindsight experience replay is the neatest of these: after a failed episode, relabel it as a success for whatever goal the agent actually achieved — it did not reach the target position, but it did reach *some* position, and that is a perfectly good training example for a goal-conditioned policy. Free data extracted from failure, and it transforms sparse goal-reaching tasks.

### 4 Learning the reward

#### REWARD MODELLING FROM PREFERENCES

Some objectives cannot be written as a function of the state. 'A helpful answer', 'a natural-looking backflip', 'a summary a person would find useful' — nobody can write these down, and a person can reliably compare two examples.

So collect pairwise preference judgements, fit a reward model to them (a Bradley–Terry model over the comparisons), and optimise the policy against the fitted model using PPO or similar. That is reinforcement learning from human feedback.

And the reward model is itself a learned, imperfect proxy — so the policy will game it, finding inputs where the model scores highly and a human would not. Which is §1's problem exactly, one level up, and it is called reward hacking — the structure is identical and recognising that is the useful part.

The standard defence is a KL penalty toward the original policy (Module 06 §2's trust region, reused for a different purpose: stay in the region where the reward model was trained) plus continued collection of fresh preferences on the current policy's outputs — which is Module 08 §2's re-fitting argument again: the exploitation generates the data that corrects it, if you keep collecting.

**WHAT TO TAKE FROM THIS MODULE**

**The reward is the specification, and specification is the hard part of this entire subject.** The algorithms of Modules 03 through 08 are well-documented and largely interchangeable; the reward is where projects actually fail.

**Every agent that did something strange was correctly optimising something you wrote. So the fix is upstream, in the objective** — not in the algorithm, not in the hyperparameters, and not in adding a penalty for the specific thing it just did, which merely relocates the exploit.

**Shape only with potential-based terms if you want the optimum preserved (§2), and otherwise know that you have deliberately changed the problem** and can say how.

**And watch what the agent actually does. Three or four reward revisions driven by observed behaviour is the normal and correct workflow** — which means **budget for it**, and treat the first reward function as a draft rather than a specification.

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                      | What it covers for this module                                                                            |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Krakovna et al. — Specification gaming examples in AI (free, maintained list) | <b>§1's examples and dozens more</b> , collected and sourced. Browse it before designing any reward.      |
| Ng, Harada & Russell — Policy Invariance Under Reward Transformations (free)  | <b>§2's theorem</b> , with the telescoping argument. Short and worth reading in full.                     |
| Andrychowicz et al. — Hindsight Experience Replay (free)                      | <b>§3's relabelling trick</b> , which is simple and unreasonably effective on sparse goal-reaching tasks. |
| Christiano et al. — Deep RL from Human Preferences (free)                     | <b>§4's method</b> , including the reward-hacking observation and the continued-collection response.      |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Read twenty entries** in the specification-gaming list and classify each by which proxy diverged.
- 2 Write a reward for a task you care about**, then spend twenty minutes finding its exploit before training.
- 3 Train against it** and report whether the agent found the same exploit or a different one.
- 4 Write down what you are not rewarding** and check each against the trained agent's behaviour.

- 5 **Add arbitrary shaping** for progress and show the optimal policy change.
- 6 **Replace it with potential-based shaping** and confirm the optimum is preserved.
- 7 **Verify the telescoping** numerically along a trajectory.
- 8 **Train on a sparse version** of the same task and report how much harder it is.
- 9 **Implement hindsight experience replay** and report the improvement on the sparse version.
- 10 **Record a video of every agent you train** and watch it. Report something you would not have seen in the curve.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give four documented specification-gaming cases and the common pattern.
- 2 Why does reinforcement learning game rewards harder than search?
- 3 Why does arbitrary shaping change the optimal policy?
- 4 State the potential-based shaping theorem and why it holds.
- 5 Give five reward-design practices.
- 6 What is the signature of specification gaming in a training curve?
- 7 Why might you choose a sparse reward, and what are the four responses?
- 8 Explain hindsight experience replay.
- 9 What is RLHF, and what is reward hacking?
- 10 What are the two standard defences against it?